

Rapport API Paoline Bouvy :

Sommaire :

[Partie 1 : sans base de donnée](#)

[Tester l'api :](#)

[Ajout d'un stockage](#)

[Partie 2 : avec une base de donnée](#)

[Mise en place de Prisma et SQLite :](#)

[Installation :](#)

[Initialiser Prisma :](#)

[Utilisation de la base de donnée :](#)

[Mes difficultés :](#)

[Source :](#)

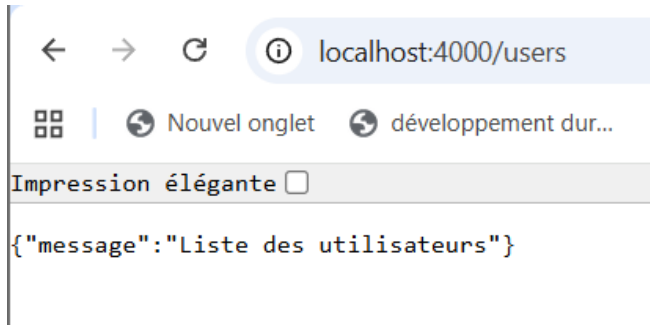
[Lien Github :](#)

Partie 1 : sans base de donnée

Tester l'api :

J'ai fait ça dans le navigateur.

```
curl -X GET http://localhost:4000/users
```



```
curl -X POST http://localhost:4000/users -H "Content-Type : application/json" -d '{"name" : "Alice", "email": "alice@example.com"}'
```

```
Cannot GET /users%20-H%20%22Content-Type%20:%20application/json%22%20-d%20'%7B%22name%22%20:%20%22Alice%22,%20%22email%22:%20%22alice@example.com%22%7D'
```

Ajout d'un stockage

```

interface User {
  id : number;
  name: string;
  email: string;
}
const users : User[] = [ //definition du tableau de User
  { id: 1, name: 'Alice', email: 'alice@example.com' },
  { id: 2, name: 'Bob', email: 'bob@example.com' }
]
export const getUsers = (req: Request, res: Response) => {
  res.json({ users}); // Envoie une réponse JSON du tableau users
};
/**
 * Contrôleur pour la route POST /users
 * Description : Ajoute un nouvel utilisateur en récupérant les données du corps de la requête
 * @param req - Objet représentant la requête HTTP contenant les données utilisateur
 * @param res - Objet permettant d'envoyer une réponse HTTP
 */
export const addUser = (req: Request, res: Response) => {
  const { name, email } = req.body; // Récupération des données envoyées dans le corps de la requête

  if(!name || !email){
    res.status(400).json({error: 'le nom et l email sont requis.'});
    return;
  }
  res.json({ message: `Utilisateur ${name} ajouté avec succès !`, email });
  // Envoie une réponse JSON confirmant l'ajout de l'utilisateur
  const newUser = {
    id: users.length+1,
    name,
    email
  };
  users.push(newUser);
  res.status(201).json(newUser);
};

```

Résultat :



Partie 2 : avec une base de donnée

J'ai fait ça avec une base de données SQLite avec PRISMA.

Mise en place de Prisma et SQLite :

Installation :

```
npm install prisma --save-dev
npm install @prisma/client
```

Initialiser Prisma :

```
npx prisma init
```

Suite à cela, on peut voir dans le projet un dossier prisma/schema.prisma puis dans le fichier .env il y a une ligne en plus.

```
DATABASE_URL="file:./dev.db";
```

On a donc comme résultat :

```
PS C:\Users\bouvy\Desktop\L3\S1\Techno Web\td_api> $env:DATABASE_URL
="file:./dev.db"; npx prisma generate
Loaded Prisma config from prisma.config.ts.
```

```
Prisma config detected, skipping environment variable loading.
Prisma schema loaded from prisma\schema.prisma
```

```
✓ Generated Prisma Client (6.19.0) to .\src\generated\prisma in 35ms
```

```
PS C:\Users\bouvy\Desktop\L3\S1\Techno Web\td_api> $env:DATABASE_URL
```

```
= "file:./dev.db"; npx prisma migrate dev --name init  
Loaded Prisma config from prisma.config.ts.
```

```
Prisma config detected, skipping environment variable loading.  
Prisma schema loaded from prisma\schema.prisma  
Datasource "db": SQLite database "dev.db" at "file:./dev.db"
```

```
SQLite database dev.db created at file:./dev.db
```

```
Applying migration `20251110141035_init`
```

Dans schema.prisma on a

```
// This is your Prisma schema file,  
// learn more about it in the docs: https://pris.ly/d/prisma-schema  
  
// Looking for ways to speed up your queries, or scale easily with your serverless or edge functions?  
// Try Prisma Accelerate: https://pris.ly/cli/accelerate-init  
  
generator client {  
  provider = "prisma-client"  
  output   = "../src/generated/prisma"  
}  
  
datasource db {  
  provider = "sqlite" //le type de base de donnée  
  url      = env("DATABASE_URL")  
}  
  
model User {  
  id Int @id @default(autoincrement())  
  name String  
  email String @unique  
}
```

Utilisation de la base de donnée :

Différentes fonctions sans user.controller.ts :

- addUser → qui permet d'ajouter un nouvel utilisateur dans la base de données
- updateUser → qui permet de mettre à jour un utilisateur par son identifiant
- getUser → permet de récupérer des données de tous les utilisateurs
- getUserById → permet de récupérer des données d'un utilisateur grâce à son identifiant
- deleteUser → permet de supprimer un utilisateur par un identifiant

Les routes dans `user.routes.ts` :

```
console.log("✅ user.routes.ts chargé");
// Importation du module Router d'Express pour gérer les routes
import { Router } from 'express';
// Importation des contrôleurs qui gèrent la logique métier pour les utilisateurs
import {
  getUsers,
  addUser,
  getUserById,
  updateUser,
  deleteUser,
} from '../controllers/user.controller';
// Création d'un routeur Express
const router = Router();

/**
 * Route GET /users
 * Description : Récupère la liste des utilisateurs
 * Contrôleur associé : getUsers (défini dans user.controller.ts)
 */
router.get('/users/:id', getUserById);
router.get('/users', getUsers);
/**
 * Route POST /users
 * Description : Ajoute un nouvel utilisateur
 * Contrôleur associé : addUser (défini dans user.controller.ts)
 */
router.post('/users', addUser);
router.put('/users/:id', updateUser);
router.delete('/users/:id', deleteUser);

// Exportation du routeur pour l'utiliser dans index.ts
export default router;
```

Mes difficultés :

Premièrement j'ai eu un problème dans mon `.env` car j'avais écrit :

```
PORT = 400;
```

alors qu'il fallait mettre :

```
PORT = 400 //sans le ;
```

Ensuite, dans la première partie, j'ai juste suivi les étapes du sujet. Dans la deuxième partie, d'abord j'ai regardé une vidéo sur les bases de données SQLite, parce que je ne savais pas comment faire.

Source :

<https://youtu.be/0My0On3lvTc?si=7DCPPuvlQeqfHaSP>

Lien Github :

https://github.com/Paoline-oss/Depot-web/upload/TD6_API